

String Handling

Q1.

Create a Java class called `InstaUsernameMasker` that takes a username string (e.g., 'insta_rockstar123') and returns a masked version where all characters except the last 4 are replaced with '*'. For example, '*****r123'.

Answer:

```
public class InstaUsernameMasker {

    public static String maskUsername(String username) {
        int length = username.length();
        if (length <= 4) {
            return username; // too short to mask meaningfully
        }

        String visiblePart = username.substring(length - 4);
        StringBuilder masked = new StringBuilder();
        for (int i = 0; i < length - 4; i++) {
            masked.append('*');
        }
        masked.append(visiblePart);

        return masked.toString();
    }

    public static void main(String[] args) {
        String username = "insta_rockstar123";
        System.out.println(maskUsername(username));
    }
}

/* Sample Output:
*****r123
*/

// Note: exact number of asterisks equals (length - 4) of the original username.
```

Q2.

Write a Java program that compares two song titles (as Strings) entered by the user and prints whether they are exactly the same using the `equals()` method, and then prints their lexicographical order using `compareTo()`.

Answer:

```
import java.util.Scanner;

public class SongTitleComparator {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter first song title: ");
        String title1 = sc.nextLine();

        System.out.print("Enter second song title: ");
        String title2 = sc.nextLine();
```

```

        if (title1.equals(title2)) {
            System.out.println("The song titles are exactly the same.");
        } else {
            System.out.println("The song titles are different.");
        }

        int result = title1.compareTo(title2);
        if (result < 0) {
            System.out.println "\"" + title1 + "\" comes before \"" + title2 + "\" lexicographically.");
        } else if (result > 0) {
            System.out.println "\"" + title1 + "\" comes after \"" + title2 + "\" lexicographically.");
        } else {
            System.out.println("Both titles are identical in lexicographical order.");
        }

        sc.close();
    }
}

/* Sample Run:
Enter first song title: Flowers
Enter second song title: Kesariya
The song titles are different.
"Flowers" comes before "Kesariya" lexicographically.
*/

```

Q3.

Build a Flipkart-style order ID generator using `StringBuilder`. Start with 'ORD' and append a random 6-digit number, then print the final order ID (e.g., ORD483921).

Answer:

```

import java.util.Random;

public class OrderIdGenerator {
    public static void main(String[] args) {
        Random rand = new Random();
        int randomNumber = 100000 + rand.nextInt(900000); // ensures exactly 6 digits

        StringBuilder orderId = new StringBuilder();
        orderId.append("ORD");
        orderId.append(randomNumber);

        System.out.println("Generated Order ID: " + orderId.toString());
    }
}

/* Sample Output (value will vary each run):
Generated Order ID: ORD483921
*/

```

Q4.

Given a long WhatsApp message string, use `substring()` and `length()` to extract and print only the first 30 characters, followed by '...' if the message is longer than 30 characters. Hint: Use `message.length()` to check the length before using `substring()`.

Answer:

```

public class MessagePreview {

```

```

public static void main(String[] args) {
    String message = "Hey! Are we still meeting for lunch today or should we reschedule for tomorrow";

    String preview;
    if (message.length() > 30) {
        preview = message.substring(0, 30) + "...";
    } else {
        preview = message;
    }

    System.out.println("Preview: " + preview);
}

/* Sample Output:
Preview: Hey! Are we still meeting for...
*/

```

Q5.

Create a Java method `maskWalletID(String walletID)` that takes a wallet ID like 'WAL12345678' and returns it in the format '***5678' (masking all but the last 4 digits). Use `StringBuffer` for the masking logic. Constraint: Do not use any external libraries or regular expressions.

Answer:

```

public class WalletMasker {

    public static String maskWalletID(String walletID) {
        int length = walletID.length();
        if (length <= 4) {
            return walletID;
        }

        String lastFour = walletID.substring(length - 4);

        StringBuffer masked = new StringBuffer();
        masked.append("***");
        masked.append(lastFour);

        return masked.toString();
    }

    public static void main(String[] args) {
        String walletID = "WAL12345678";
        System.out.println(maskWalletID(walletID));
    }

    /* Sample Output:
    ***5678
    */
}

```